

Mục Lục

TÀI LIỆU BÁO CÁO KỸ THUẬT: KIẾN TRÚC & TRIỂN KHAI DOCKER HỆ THỐNG PHARMACY [1](#tài-liệu-báo-cáo-kỹ-thuật-kiến-trúc-triển-khai-docker-hệ-thống-pharmacy)

1. THÔNG TIN TỔNG QUAN HỆ THỐNG [1](#thông-tin-tổng-quan-hệ-thống)

2. CẤU TRÚC THƯ MỤC DỰ ÁN (PROJECT STRUCTURE) [2](#cấu-trúc-thư-mục-dự-án-project-structure)

3. CHI TIẾT FILE CẤU HÌNH & GIẢI THÍCH (CONFIGURATION FILES) [3](#chi-tiết-file-cấu-hình-giải-thích-configuration-files)

A. File cấu hình môi trường mẫu: .env.example [3](#a.-file-cấu-hình-môi-trường-mẫu-.env.example)

B. Cấu hình Frontend Dockerfile: frontend/Dockerfile [4](#b.-cấu-hình-frontend-dockerfile-frontenddockerfile)

C. Cấu hình Nginx Custom: frontend/nginx.conf [5](#c.-cấu-hình-nginx-custom-frontendnginx.conf)

D. Cấu hình Backend Dockerfile: backend/Dockerfile [6](#d.-cấu-hình-backend-dockerfile-backenddockerfile)

E. File Docker Compose gốc: docker-compose.yml [7](#e.-file-docker-compose-gốc-docker-compose.yml)

F. File Docker Compose Phát triển: docker-compose.dev.yml [9](#f.-file-docker-compose-phát-triển-docker-compose.dev.yml)

G. File Docker Compose Sản xuất: docker-compose.prod.yml [10](#g.-file-docker-compose-sản-xuất-docker-compose.prod.yml)

4. KIẾN TRÚC & TRIỂN KHAI QUA 3 MÔI TRƯỜNG [11](#kiến-trúc-triển-khai-qua-3-môi-trường)

A. Sơ đồ Luồng Dữ liệu & Điều phối Mạng Container [11](#a.-sơ-đồ-luồng-dữ-liệu-điều-phối-mạng-container)

B. Vận Hành Hệ Thống Trên 3 Môi Trường Khác Nhau (Dev, UAT, Prod) [12](#b.-vận-hành-hệ-thống-trên-3-môi-trường-khác-nhau-dev-uat-prod)

5. HƯỚNG DẪN TRIỂN KHAI [15](#hướng-dẫn-triển-khai)

BƯỚC 1: Chuẩn bị môi trường hệ thống [15](#bước-1-chuẩn-bị-môi-trường-hệ-thống)

BƯỚC 2: Khởi chạy hệ thống bằng Docker Compose [15](#bước-2-khởi-chạy-hệ-thống-bằng-docker-compose)

BƯỚC 3: Cài đặt Tool và Nhập dữ liệu mẫu vào MongoDB [17](#bước-3-cài-đặt-tool-và-nhập-dữ-liệu-mẫu-vào-mongodb)

BƯỚC 4: Kiểm tra giao diện người dùng (Frontend -> Backend) [18](#bước-4-kiểm-tra-giao-diện-người-dùng-frontend-backend)

TÀI LIỆU BÁO CÁO KỸ THUẬT: KIẾN TRÚC & TRIỂN KHAI DOCKER HỆ THỐNG PHARMACY

1. THÔNG TIN TỔNG QUAN HỆ THỐNG

Hệ thống được thiết kế theo mô hình **Client-Server (3-Tier Architecture)** phân tách rõ ràng các tầng giao diện, xử lý nghiệp vụ và lưu trữ dữ liệu nhằm đảm bảo tính cô lập, dễ bảo trì và khả năng mở rộng.

- **Kiến trúc:** Mô hình Client-Server hoạt động trong môi trường Dockerized.
- **Frontend (Tầng Giao Diện):**
 - **Công nghệ:** ReactJS tích hợp Vite (Build tool) và styling bằng TailwindCSS.
 - **Máy chủ Web tĩnh:** Nginx (được cấu hình làm web server phục vụ file tĩnh và Reverse Proxy cho API).
- **Backend (Tầng Nghiệp Vụ):**
 - **Công nghệ:** Java Spring Boot (sử dụng Maven để quản lý thư viện và vòng đời build).
 - **Phiên bản runtime:** Java 17 (Eclipse Temurin JRE).
- **Database (Tầng Lưu Trữ):**
 - **Công nghệ:** MongoDB 8.0 (Hệ quản trị cơ sở dữ liệu NoSQL).
- **Containerization (Đóng gói & Điều phối):**
 - **Công nghệ:** Docker Engine và Docker Compose.
 - **Môi trường vận hành hỗ trợ:** Phát triển cục bộ (Development) và Triển khai thực tế (Production).

2. CẤU TRÚC THƯ MỤC DỰ ÁN (PROJECT STRUCTURE)

Dưới đây là sơ đồ cây thư mục của dự án **Pharmacy Project**, thể hiện rõ vị trí của mã nguồn Frontend, Backend cùng các file cấu hình Docker, biến môi trường và script vận hành:

pharmacy-project/	
— .env.example	# File môi trường mẫu (chứa các key cấu hình không có giá trị nhạy cảm)
— docker-compose.yml	# File Docker Compose gốc (định nghĩa services, networks và volumes)
— docker-compose.dev.yml	# File Docker Compose ghi đề cho môi trường Phát triển (Development)
— docker-compose.prod.yml	# File Docker Compose ghi đề cho môi trường Vận hành (Production)
— import_db.sh	# Script Bash nạp dữ liệu mẫu vào MongoDB container
— README.md	# Hướng dẫn tổng quan dự án
— backend/	# Mã nguồn tầng Backend (Java Spring Boot)
— pom.xml	# File quản lý dependencies của Maven
— Dockerfile	# File cấu hình build/run Docker image cho Backend (Multi-stage)
— src/	# Thư mục chứa mã nguồn Java
— main/	
— test/	
— target/	# Thư mục build output (file .jar) của Maven (bị bỏ qua khi push)
— frontend/	# Mã nguồn tầng Frontend (ReactJS + Vite)
— package.json	# File khai báo dependencies và scripts của Node.js
— Dockerfile	# File cấu hình build/run Docker image cho Frontend (Multi-stage)
— nginx.conf	# File cấu hình máy chủ Nginx (Routing SPA và Reverse Proxy)
— vite.config.js	# File cấu hình Vite build tool
— tailwind.config.js	# File cấu hình TailwindCSS
— index.html	# Điểm nhập giao diện HTML chính
— src/	# Thư mục chứa mã nguồn component và logic React
— json_data/	# Dữ liệu JSON mẫu phục vụ việc import ban đầu
— employees.json	
— products.json	
— customers.json	
— invoices.json	

3. CHI TIẾT FILE CẤU HÌNH & GIẢI THÍCH (CONFIGURATION FILES)

Trong phần này, toàn bộ nội dung của các file cấu hình cốt lõi sẽ được hiển thị chi tiết cùng phần cắt nghĩa kỹ thuật cho từng dòng hoặc khối lệnh quan trọng.

A. File cấu hình môi trường mẫu: .env.example

File này chứa danh sách các biến môi trường cấu hình cho hệ thống nhưng không đi kèm các giá trị thực tế nhạy cảm (như mật khẩu sản xuất). Khi triển khai, quản trị viên sẽ copy file này thành .env và điền giá trị phù hợp.

```
# Cấu hình container frontend
FRONTEND_IMAGE_TAG=
```

```
# Cấu hình container backend
SPRING_BOOT_IMAGE_TAG=
MONGODB_URI=
MONGODB_DATABASE=
```

```
# Cấu hình container mongodb
MONGO_INITDB_ROOT_USERNAME=
MONGO_INITDB_ROOT_PASSWORD=
MONGO_INITDB_DATABASE=
```

Giải thích chi tiết:

- FRONTEND_IMAGE_TAG: Định nghĩa tag phiên bản cho Docker Image của Frontend (ví dụ: 2.0 hoặc latest).
- SPRING_BOOT_IMAGE_TAG: Định nghĩa tag phiên bản cho Docker Image của Backend Spring Boot.
- MONGODB_URI: Chuỗi kết nối từ Backend sang Database MongoDB (ví dụ: mongodb://root:root@mongodb:27017/pharmacy?authSource=admin).
- MONGODB_DATABASE: Tên database mặc định mà Backend Spring Boot sẽ sử dụng để đọc ghi dữ liệu.
- MONGO_INITDB_ROOT_USERNAME & MONGO_INITDB_ROOT_PASSWORD: Tài khoản quản trị tối cao (Root Administrator) được khởi tạo tự động khi container MongoDB chạy lần đầu tiên.
- MONGO_INITDB_DATABASE: Tên cơ sở dữ liệu được Docker tự động tạo lúc khởi tạo MongoDB Engine.

B. Cấu hình Frontend Dockerfile: frontend/Dockerfile

Dockerfile của Frontend sử dụng phương pháp **Multi-stage Build** để chia quy trình đóng gói thành hai bước độc lập: Build mã nguồn React và Chạy ứng dụng tĩnh bằng Nginx.

```
# Bước 1: Build mã nguồn React sử dụng Node.js
FROM node:20-alpine AS build
```

```
WORKDIR /app
```

```
# Copy package.json và package-lock.json để cài đặt dependencies trước (tối ưu cache Docker)
COPY package*.json ./
```

RUN npm install

Copy toàn bộ mã nguồn frontend và build bản production

COPY . .

RUN npm run build

Bước 2: Dùng Nginx để chạy bản build

FROM nginx:alpine

Copy file cấu hình Nginx custom vào container

COPY nginx.conf /etc/nginx/conf.d/default.conf

Copy thư mục build từ bước 1 vào thư mục chứa static files của Nginx

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Giải thích chi tiết:

- FROM node:20-alpine AS build: Khởi tạo giai đoạn build (Stage 1) sử dụng môi trường Node.js phiên bản 20 trên nền hệ điều hành Alpine Linux siêu nhẹ nhằm tiết kiệm bộ nhớ. Giai đoạn này được đặt danh danh (alias) là build.
- COPY package*.json ./ & RUN npm install: Copy các file định nghĩa thư viện trước rồi mới cài đặt. Quy trình này giúp tận dụng tối đa cơ chế lưu cache layer của Docker; nếu package.json không thay đổi, Docker sẽ bỏ qua bước tải thư viện ở các lần build sau, giảm thời gian build từ vài phút xuống vài giây.
- RUN npm run build: Biên dịch mã nguồn React (JSX, ES6, CSS) thành các file tĩnh (HTML, JS, CSS) trong thư mục /app/dist.
- FROM nginx:alpine: Bắt đầu giai đoạn chạy (Stage 2). Docker sẽ vứt bỏ toàn bộ môi trường Node.js và các file source code nặng nề ở bước 1, chỉ lấy một image Nginx Alpine siêu nhẹ (chỉ khoảng 20-30MB) làm runtime nền tảng.
- COPY nginx.conf /etc/nginx/conf.d/default.conf: Ghi đè file cấu hình mặc định của Nginx bằng file cấu hình custom của dự án để hỗ trợ routing cho Single Page Application (SPA) và reverse proxy.
- COPY --from=build /app/dist /usr/share/nginx/html: Sao chép kết quả build tĩnh (/app/dist ở Stage 1) vào thư mục phân phối file tĩnh của Nginx (/usr/share/nginx/html).
- EXPOSE 80: Khai báo container sẽ lắng nghe yêu cầu kết nối ở cổng
- CMD ["nginx", "-g", "daemon off;"]: Lệnh khởi chạy Nginx dưới dạng tiến trình chính ở foreground. Nếu Nginx dừng, container sẽ dừng theo.

C. Cấu hình Nginx Custom: frontend/nginx.conf

File cấu hình này giải quyết hai vấn đề sống còn của ứng dụng React: routing phía client và định tuyến API gọi đến Backend.

```
server {
    listen 80;
    server_name localhost;

    # Thư mục chứa các file static đã build của React
    root /usr/share/nginx/html;
    index index.html;

    # Xử lý Routing SPA (React Router)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Proxy ngược (Reverse Proxy) các request API sang container Spring Boot
    location /api {
        proxy_pass http://spring-boot:8081;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Cache cấu hình cho assets tĩnh
    location ~* \.(?:css|js|jpg|jpeg|gif|png|ico|cur|gz|svg|svgz|mp4|ogg|ogv|
webm|htc)$ {
        expires 1M;
        access_log off;
        add_header Cache-Control "public";
    }
}
```

Giải thích chi tiết:

- `try_files $uri $uri/ /index.html;`: Khi client truy cập các route ảo của React Router (ví dụ: `/dashboard`, `/invoices`), Nginx sẽ không tìm thấy các thư mục vật lý tương ứng trên đĩa cứng. Dòng này yêu cầu Nginx chuyển hướng tất cả các request không tìm thấy file về lại `index.html` để React Router tự xử lý routing phía client, tránh lỗi 404 Not Found.
- `location /api`: Đánh dấu các request bắt đầu bằng `/api/` (ví dụ: client gọi API lấy danh sách thuốc tại `http://localhost/api/products`).
- `proxy_pass http://spring-boot:8081;`: Cơ chế **Reverse Proxy**. Nginx sẽ chuyển tiếp request API này đến container backend chạy Spring Boot tại địa chỉ nội bộ `http://spring-boot:8081` trong mạng Docker.

- `proxy_set_header X-Real-IP $remote_addr`: Truyền địa chỉ IP thực của client về cho backend ghi log hoặc xử lý bảo mật (nếu không có, backend chỉ nhận được IP nội bộ của Nginx container).

D. Cấu hình Backend Dockerfile: `backend/Dockerfile`

Tương tự Frontend, Backend Spring Boot cũng được đóng gói qua **Multi-stage Build** nhằm giảm kích thước image từ hàng GB (chứa Maven và JDK đầy đủ) xuống chỉ khoảng 150MB (chỉ chứa JRE).

```
# stage 1
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /build

COPY pom.xml .
RUN mvn dependency:go-offline -B

COPY src ./src
RUN mvn package -DskipTests -B

# stage 2
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=builder /build/target/*.jar app.jar

EXPOSE 8081

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Giải thích chi tiết:

- `FROM maven:3.9-eclipse-temurin-17 AS builder`: Giai đoạn build (Stage 1). Sử dụng image Maven chính thức tích hợp sẵn OpenJDK 17 (Temurin phân phối) để tiến hành biên dịch ứng dụng Java.
- `RUN mvn dependency:go-offline -B`: Tải trước toàn bộ các file thư viện `.jar` được khai báo trong `pom.xml` về local repository của container. Tham số `-B` (Batch-mode) tắt bớt log tải file rườm rà.
- `RUN mvn package -DskipTests -B`: Biên dịch mã nguồn Java và đóng gói thành file thực thi duy nhất `.jar`. Cờ `-DskipTests` bỏ qua việc chạy unit test để tăng tốc độ đóng gói.
- `FROM eclipse-temurin:17-jre`: Giai đoạn chạy (Stage 2). Sử dụng image Java Runtime Environment (JRE) chính thức, lược bỏ trình biên dịch JDK và Maven để giảm dung lượng đĩa tối đa và tăng tính an toàn (giảm bề mặt tấn công bảo mật).
- `COPY --from=builder /build/target/*.jar app.jar`: Sao chép file `.jar` đã build thành công từ Stage 1 sang Stage 2 và đặt tên ngắn gọn là `app.jar`.

- ENTRYPOINT ["java", "-jar", "app.jar"]: Lệnh cố định để chạy file JAR khi container bắt đầu khởi động.

E. File Docker Compose gốc: docker-compose.yml

File này đóng vai trò như bản vẽ quy hoạch kiến trúc điều phối, liên kết các thành phần Frontend, Backend và Database vào một hệ thống thống nhất.

```
services:
  frontend:
    image: vokhoaecho/pharmacy-frontend:${FRONTEND_IMAGE_TAG}
    container_name: pharmacy-frontend
    depends_on:
      - spring-boot
    networks:
      - frontend-net

  spring-boot:
    image: dinhhsi1401/pharmacy:${SPRING_BOOT_IMAGE_TAG}
    container_name: pharmacy-backend
    environment:
      MONGODB_URI: ${MONGODB_URI}
      MONGODB_DATABASE: ${MONGODB_DATABASE}
    depends_on:
      mongodb:
        condition: service_healthy
    networks:
      - frontend-net
      - backend-net

  mongodb:
    image: mongo:8
    container_name: pharmacy-mongo
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_INITDB_ROOT_USERNAME}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
      MONGO_INITDB_DATABASE: ${MONGO_INITDB_DATABASE}
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s
    volumes:
      - mongo-data:/data/db
    networks:
      - backend-net

volumes:
```


mongo-data:

networks:

frontend-net:

backend-net:

Giải thích chi tiết:

- image: vokhoaecho/pharmacy-frontend:\${FRONTEND_IMAGE_TAG}: Docker Compose sẽ lấy giá trị biến môi trường FRONTEND_IMAGE_TAG trong file .env để kéo đúng phiên bản image từ Docker Hub về chạy.
- depends_on:
 - frontend phụ thuộc spring-boot: Đảm bảo backend được khởi tạo trước frontend.
 - spring-boot phụ thuộc mongodb với điều kiện service_healthy: Backend sẽ không khởi chạy cho đến khi tiến trình kiểm tra sức khỏe (healthcheck) của MongoDB báo trạng thái “khỏe mạnh” (đã sẵn sàng nhận kết nối). Việc này ngăn chặn lỗi crash của Spring Boot do cố kết nối vào Database khi DB chưa khởi động xong.
- healthcheck: Đoạn mã chạy thử lệnh mongosh --eval "db.adminCommand('ping')" định kỳ 10 giây/lần bên trong container MongoDB để xác nhận Database đã hoàn toàn hoạt động bình thường.
- volumes: - mongo-data:/data/db: Ánh xạ vùng dữ liệu /data/db của container vào volume mongo-data do Docker quản lý trên máy Host. Điều này đảm bảo dữ liệu của nhà thuốc được lưu trữ bền vững (không bị mất đi khi xóa container).
- networks: Phân chia hệ thống thành hai mạng ảo cô lập:
 - frontend-net: Nơi frontend và spring-boot nói chuyện với nhau.
 - backend-net: Nơi spring-boot truy vấn dữ liệu từ mongodb.
 - *Mục đích bảo mật*: Container frontend hoàn toàn không có kết nối tới backend-net, nghĩa là kẻ tấn công từ bên ngoài nếu chiếm quyền kiểm soát được container frontend cũng không thể quét mạng hay tấn công trực tiếp vào cơ sở dữ liệu MongoDB.

F. File Docker Compose Phát triển: docker-compose.dev.yml

File cấu hình bổ sung (override) này dùng để cấu hình cổng kết nối công khai khi chạy trên môi trường phát triển cục bộ (Development) của lập trình viên.

services:

frontend:

ports:

- "8082:80"

restart: unless-stopped

map port Spring ra ngoài để test api

spring-boot:

```

ports:
  - "8081:8081"
restart: unless-stopped

# map port Mongo ra ngoài để dùng DB Tool (DbSchema, Compass...) kiểm tra
mongodb:
  ports:
    - "27017:27017"
  restart: unless-stopped

```

Giải thích chi tiết:

- ports: - "8082:80": Ánh xạ cổng 8082 của máy Host vào cổng 80 của container Frontend. Lập trình viên có thể truy cập giao diện qua địa chỉ `http://localhost:8082`.
- ports: - "8081:8081": Mở cổng API của Backend ra ngoài máy Host để kiểm thử các endpoint bằng các công cụ độc lập như Postman.
- ports: - "27017:27017": Mở cổng cơ sở dữ liệu để dev sử dụng các phần mềm quản trị trực quan như MongoDB Compass hoặc DbSchema kết nối trực tiếp vào DB đang chạy trong Docker.

G. File Docker Compose Sản xuất: `docker-compose.prod.yml`

File cấu hình bổ sung này ghi đè các cấu hình nhằm phục vụ môi trường vận hành thực tế (Production), tối ưu hóa tài nguyên phần cứng và tăng cường bảo mật.

```

services:
  frontend:
    restart: unless-stopped
    ports:
      - "80:80"
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 256M

  spring-boot:
    restart: unless-stopped
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

```

```

    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 768M

mongodb:
  restart: unless-stopped
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: 1G

```

Giải thích chi tiết:

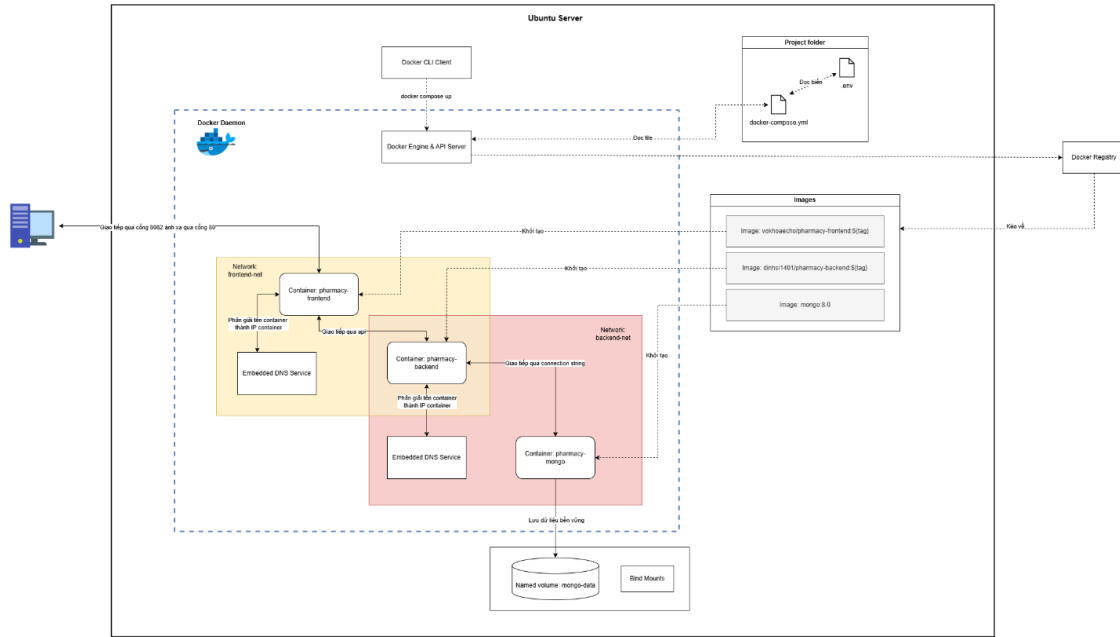
- `ports: - "80:80"`: Ánh xạ cổng chuẩn HTTP (80) ra ngoài internet. Người dùng chỉ cần truy cập địa chỉ IP của server hoặc domain mà không cần chỉ định cổng.
- **Không mở cổng Backend (8081) và Database (27017)**: Trong production, hai dịch vụ này được ẩn hoàn toàn khỏi internet. Frontend Nginx sẽ lo việc proxy API vào trong mạng nội bộ Docker. Điều này loại bỏ hoàn toàn các cuộc tấn công quét cổng từ bên ngoài vào database và server nghiệp vụ.
- `logging`: Giới hạn log ghi ra đĩa cứng của mỗi container tối đa 10 Megabytes (10m) và lưu tối đa 3 file xoay vòng (`max-file: "3"`). Việc này ngăn ngừa lỗi máy chủ bị hết dung lượng đĩa do log hệ thống ghi vô tội vạ qua nhiều tháng hoạt động.
- `deploy.resources.limits`: Giới hạn phần cứng tối đa mà container được phép sử dụng (CPU và RAM). Việc giới hạn này ngăn ngừa trường hợp một service bị lỗi rò rỉ bộ nhớ (memory leak) tiêu thụ sạch tài nguyên của máy chủ vật lý, làm sập toàn bộ các dịch vụ khác chạy chung.

4. KIẾN TRÚC & TRIỂN KHAI QUA 3 MÔI TRƯỜNG

Phần này phân tích sâu hơn các giải pháp kiến trúc đã được áp dụng trong Pharmacy Project nhằm giải quyết các bài toán vận hành thực tế của các ứng dụng doanh nghiệp.

A. Sơ đồ Luồng Dữ liệu & Điều phối Mạng Container

Sơ đồ dưới đây biểu diễn cách thức các request từ ngoài internet đi vào hệ thống qua Nginx và được định tuyến đến các phân vùng mạng an toàn bên trong Docker Engine:



Ảnh có chứa văn bản, ảnh chụp màn hình, biểu đồ, Song song Nội dung do AI tạo ra có thể không chính xác.

B. Vận Hành Hệ Thống Trên 3 Môi Trường Khác Nhau (Dev, UAT, Prod)

1. Tổng quan về vận hành 3 môi trường

Trong quy trình phát triển phần mềm, một ứng dụng bắt buộc phải trải qua nhiều môi trường vận hành trước khi tới tay người dùng cuối: - **Dev (Development)**: Cần sự linh hoạt, bật hot-reload, map port trực tiếp để lập trình viên test API và debug. - **UAT (User Acceptance Testing)**: Môi trường kiểm thử giả lập hệ thống thật với cơ sở dữ liệu độc lập. - **Prod (Production)**: Môi trường thực tế, đặt nặng yếu tố an toàn - bảo mật - hiệu năng (đóng port kín, giới hạn CPU/RAM, bật chính sách tự khởi động lại).

Thách thức đặt ra là làm sao cấu hình hệ thống đồng bộ cho cả 3 môi trường nhưng vẫn đảm bảo tính cách ly và bảo mật riêng biệt.

2. Phương pháp và lý do lựa chọn

Thay vì viết 3 file Docker Compose độc lập gây lộn xộn cấu hình, báo cáo này áp dụng kỹ thuật **Kế thừa & Ghi đè (Multiple Compose Files)** kết hợp **Cách ly biến môi trường (.env)** vì những lợi ích sau: 1. **Tối ưu mã nguồn (Nguyên lý DRY)**: Tách toàn bộ cấu hình chung (tên dịch vụ, network nội bộ, volume) vào file base docker-compose.yml, loại bỏ việc sao chép lặp lại code. 2. **Linh hoạt và Dễ bảo trì**: Các file môi trường (docker-compose.dev.yml, docker-compose.prod.yml) chỉ cần viết ngắn gọn để ghi đè các thông số đặc thù. Khi thêm dịch vụ mới, chỉ cần sửa một nơi tại file base. 3. **Bảo mật dữ liệu nhạy cảm**: Tách biệt “khung xương” hệ thống (file YAML) khỏi “thông tin thực tế” (mật khẩu, API key) qua các file .env riêng biệt. Các file cấu hình có thể push lên GitHub, còn tệp chứa mật khẩu thật (.env.prod) sẽ được giữ bảo mật trên server. 4. **Hỗ trợ tự động hóa (CI/CD)**:

Cú pháp gọi chuỗi file `-f` và `--env-file` giúp các công cụ tự động (GitHub Actions, GitLab CI) dễ dàng nhận diện môi trường để deploy tự động.

3. Kỹ thuật kế thừa và ghi đè cấu hình của Docker Compose

Thay vì viết 3 file độc lập gây lộn xộn, ta có thể áp dụng kỹ thuật kế thừa và ghi đè cấu hình (Override) của Docker Compose bằng cách viết một file compose cơ sở (base) chứa thông tin chung và đối với mỗi môi trường sẽ có một file chứa thông tin riêng để override: 1.

docker-compose.yml (file base): Định nghĩa khung xương hệ thống, tên container, volume, network và các biến template. 2. **docker-compose.dev.yml (file override môi trường dev):** Map port trực tiếp ra ngoài để tiện test, bật hot-reload, ... 3. **docker-compose.uat.yml (file override môi trường uat):** Cấu hình tương tự Prod nhưng sử dụng cơ sở dữ liệu test riêng biệt. 4. **docker-compose.prod.yml (file override môi trường prod):** Bỏ map port trực tiếp, giới hạn RAM/CPU, bật restart policy, cấu hình proxy bảo mật, ...

Khi tiến hành triển khai ứng dụng, tùy thuộc vào máy chủ hiện tại thuộc môi trường nào, quản trị viên sử dụng cú pháp kết hợp file tương ứng, ví dụ môi trường dev:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d
```

Lưu ý: Khi gọi nhiều file `-f` liên tiếp, Docker Compose sẽ ưu tiên đọc các file đứng sau để ghi đè (override) các giá trị trùng lặp của các file đứng trước. File `docker-compose.yml` (base) luôn phải được gọi đầu tiên làm nền móng.

4. Quản lý biến môi trường (.env) cách ly

Không nên thiết lập cứng các giá trị của các biến môi trường trong file `docker-compose.yml` mà phải tách giá trị ra một file `.env` để không bị xung đột giữa các môi trường (vì mỗi môi trường sử dụng giá trị khác nhau) và đảm bảo các giá trị nhạy cảm không bị lộ ra ngoài (vì file `.env` sẽ không commit lên GitHub). Ta chỉ commit file `.env.example` để khai báo các biến cần sử dụng mà không điền giá trị. Mỗi môi trường sẽ có một file `.env` tương ứng:

1. **.env.example:** File mẫu chứa danh sách tên các biến cần thiết cho dự án nhưng không bao gồm giá trị cụ thể. File này được phép commit lên GitHub để các thành viên khác biết dự án cần những cấu hình nào để khởi chạy. 2. **.env.dev:** File chứa các giá trị cấu hình cụ thể phục vụ cho môi trường phát triển (Development) cục bộ của lập trình viên (ví dụ: kết nối tới database local, bật chế độ debug, port chạy thử nghiệm). File này thường được giữ dưới máy cá nhân và không commit lên Git. 3. **.env.prod hoặc .env.uat:** File chứa các thông số cấu hình thực tế, bảo mật cao dành riêng cho môi trường vận hành (Production/UAT) như: mật khẩu database thật, API key của bên thứ ba, chứng chỉ bảo mật. File này tuyệt đối không được commit lên GitHub mà sẽ được cấu hình trực tiếp trên server hoặc thông qua các công cụ CI/CD.

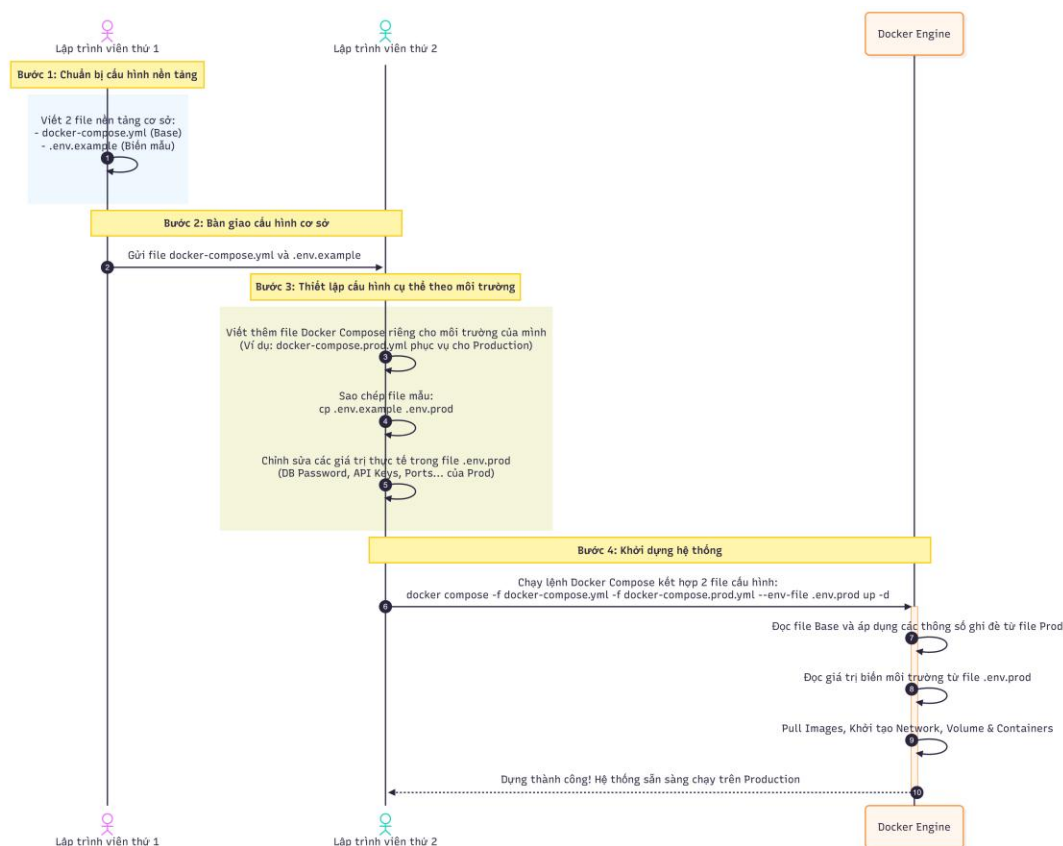
Khi triển khai, ví dụ với môi trường dev, lập trình viên copy file `.env.example` thành file `.env.dev`, điền các thông số local rồi chạy lệnh:

```
docker compose --env-file .env.dev up -d
```

Mặc định Docker Compose sẽ đọc file `.env` nằm cùng thư mục với file `docker-compose.yml`, lệnh trên sẽ chỉ định file `.env.dev` để đọc và truyền giá trị biến.

5. Sơ đồ luồng hoạt động đa môi trường

Dưới đây là sơ đồ mô tả luồng hoạt động và quy trình điều phối cấu hình Docker Compose trên 3 môi trường khác nhau:



Sơ đồ luồng vận hành Docker Compose đa môi trường

5. HƯỚNG DẪN TRIỂN KHAI

Để triển khai hệ thống thành công, vui lòng thực hiện tuần tự các bước dưới đây.

BƯỚC 1: Chuẩn bị môi trường hệ thống

Đảm bảo máy chủ hoặc máy tính cá nhân đã cài đặt sẵn Docker Engine và Docker Compose CLI.

- Mở terminal (trên Linux) chạy hai lệnh kiểm tra:
- `docker --version`
- `docker compose version`

```
khoavo@ovl-iiuh:~$ docker --version
docker compose version
Docker version 29.6.0, build fb59821
Docker Compose version v5.2.0
```

Ảnh có chứa văn bản, Phông chữ, ảnh chụp màn hình Nội dung do AI tạo ra có thể không chính xác.

BƯỚC 2: Khởi chạy hệ thống bằng Docker Compose

Tạo file cấu hình môi trường .env và kích hoạt toàn bộ các container dịch vụ hoạt động ở chế độ chạy ngầm.

1. Sao chép cấu hình môi trường mẫu:

- `cp .env.example .env`

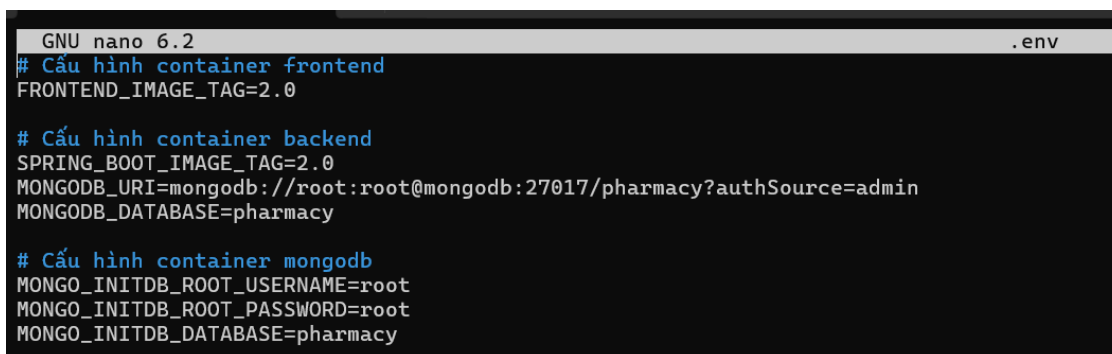
2. Mở file .env bằng trình soạn thảo (ví dụ: `nano .env`) và điền các giá trị tag

ví dụ:

```
# Cấu hình container frontend
FRONTEND_IMAGE_TAG``=``2.0
```

```
# Cấu hình container backend
SPRING_BOOT_IMAGE_TAG``=``2.0
MONGODB_URI``=``mongodb://root:root@mongodb:27017/pharmacy``?``authSource=admin
MONGODB_DATABASE``=``pharmacy
```

```
# Cấu hình container mongodb
MONGO_INITDB_ROOT_USERNAME``=``root
MONGO_INITDB_ROOT_PASSWORD``=``root
MONGO_INITDB_DATABASE``=``pharmacy
```



```
GNU nano 6.2 .env
# Cấu hình container frontend
FRONTEND_IMAGE_TAG=2.0

# Cấu hình container backend
SPRING_BOOT_IMAGE_TAG=2.0
MONGODB_URI=mongodb://root:root@mongodb:27017/pharmacy?authSource=admin
MONGODB_DATABASE=pharmacy

# Cấu hình container mongodb
MONGO_INITDB_ROOT_USERNAME=root
MONGO_INITDB_ROOT_PASSWORD=root
MONGO_INITDB_DATABASE=pharmacy
```

Ảnh có chứa văn bản, ảnh chụp màn hình, Phông chữ, phần mềm Nội dung do AI tạo ra có thể không chính xác.

3. Khởi chạy hệ thống trên môi trường phát triển cục bộ bằng cách gộp file compose gốc và file dev:

- `sudo docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build`

```
khaoVo@ool-juh:~/pharmacy-project$ sudo docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
[+] up 5/5
✓Network pharmacy-project_backend-net Created 0.1s
✓Network pharmacy-project_frontend-net Created 0.1s
✓Container pharmacy-mongo Healthy 9.3s
✓Container pharmacy-backend Started 8.8s
✓Container pharmacy-frontend Started 9.5s
```

4. Kiểm tra trạng thái hoạt động của các container:

- `sudo docker ps`

BƯỚC 3: Cài đặt Tool và Nhập dữ liệu mẫu vào MongoDB

Thực hiện cài đặt công cụ cần thiết vào container database và chạy script nạp dữ liệu.

1. Truy cập vào container `pharmacy-mongo` dưới quyền root để cài đặt công cụ import:

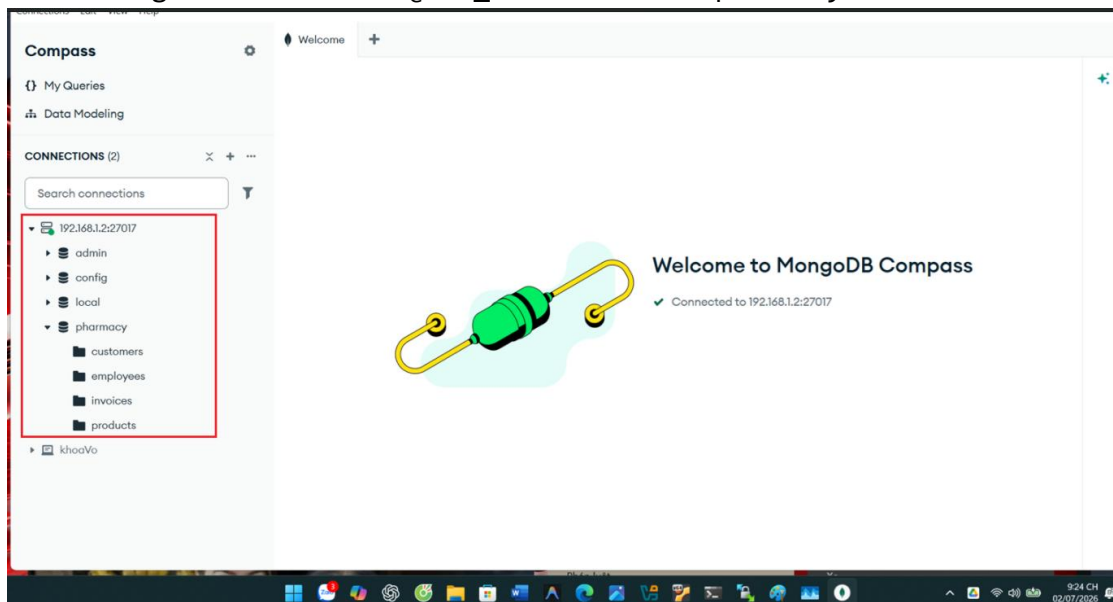
- `sudo docker exec -u 0 -it pharmacy-mongo apt-get update`
`sudo docker exec -u 0 -it pharmacy-mongo apt-get install -y mongodb-database-tools`

2. Thực thi script import dữ liệu mẫu từ thư mục gốc của dự án trên máy host:

- `bash import_db.sh`

3. Sử dụng công cụ **MongoDB Compass** trên máy tính cá nhân kết nối vào cơ sở dữ liệu theo URI:

`mongodb://root:root@<IP_Server>:27017/pharmacy?authSource=admin`

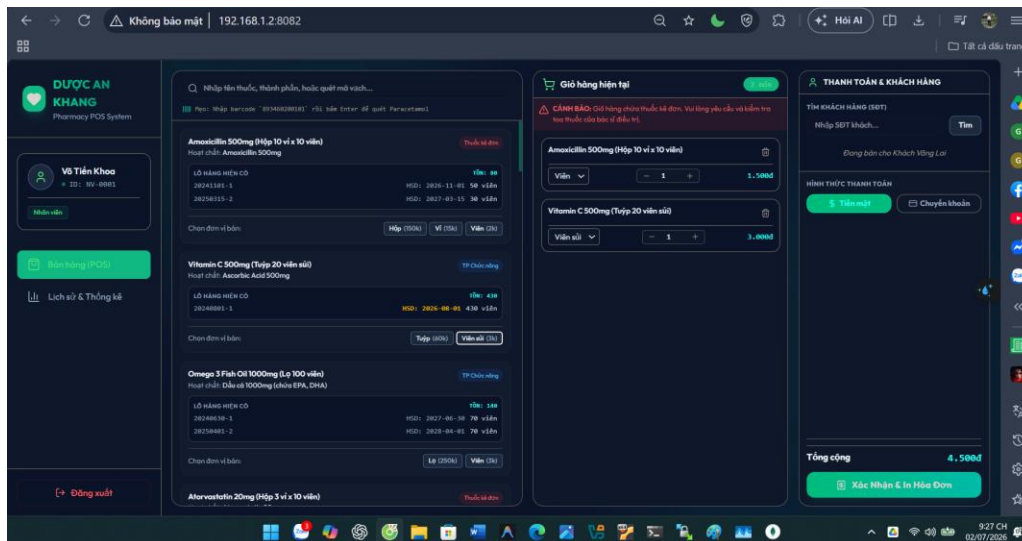


Ảnh có chứa văn bản, phần mềm, Biểu tượng máy tính, máy tính Nội dung do AI tạo ra có thể không chính xác.

BƯỚC 4: Kiểm tra giao diện người dùng (Frontend -> Backend)

Xác thực luồng hoạt động khép kín (End-to-End) của ứng dụng từ giao diện người dùng tới database.

1. Mở trình duyệt Web (Chrome, Edge hoặc Firefox) truy cập địa chỉ:
`http://<IP_Server>:8082`.
2. Đăng nhập hệ thống với tài khoản nhân viên mẫu:
 - **Username:** NV-0001
 - **Password:** Votienkhoa123@
3. Thực hiện thao tác thêm mới một sản phẩm thuốc vào giỏ hàng hoặc tạo đơn hàng mới.



Ảnh có chứa văn bản, ảnh chụp màn hình, phần mềm, Phần mềm đa phương tiện Nội dung do AI tạo ra có thể không chính xác.



Ảnh có chứa văn bản, ảnh chụp màn hình, phần mềm, đồ điện tử Nội dung do AI tạo ra có thể không chính xác.